

App consent grant investigation

garrodonnell

25–32 minutes

This article provides guidance on identifying and investigating app consent attacks, protecting information, and minimizing further risks.

This article contains the following sections:

- **Prerequisites:** Covers the specific requirements you need to complete before starting the investigation. For example, logging that should be turned on, roles and permissions required, among others.
- **Workflow:** Shows the logical flow that you should follow to perform this investigation.
- **Checklist:** Contains a list of tasks for each of the steps in the flow chart. This checklist can be helpful in highly regulated environments to verify steps taken or simply as a quality gate for yourself.
- **Investigation steps:** Includes a detailed step-by-step guidance for this specific investigation.
- **Recovery:** Contains high level steps on how to recover/mitigate from an Illicit Application Consent grant attack.
- **References:** Contains more reading and reference materials.

Here are general settings and configurations you should

complete when performing an investigation for Application Consent Grants. Before starting the investigation, make sure read about the types of consent permissions explained in [Consent permission types](#).

To start the investigation process, you need the following data:

- Detail of indicators of compromise (IoCs)
- The date and time when you noticed the incident
- Date range
- Number of compromised accounts
- Name of compromised accounts
- Roles of the compromised account
- Are the accounts highly privileged (GA Microsoft Exchange, SharePoint)?
- Are there any Enterprise Applications that are related to the incident?
- Did users report any applications that were requesting permissions to data on their behalf?

Ensure you complete the following installations and configuration requirements:

- The Microsoft.Graph PowerShell module is installed.
- You have [Global Administrator](#) rights on the tenant that the script run against.
- You're assigned local administrator role on the computer that you use to run the scripts.

Use this command to install the Microsoft.Graph module.

```
Install-Module -Name Microsoft.Graph -Verbose
```

Note

If you're prompted to install the modules from an untrusted repository, type **Y** and press **Enter**.

1. Download the [Get-AzureADPSPPermissions.ps1](#) script from GitHub to a folder from which you run the script. The output file "*permissions.csv*" is also be written to this same folder.
2. Open a PowerShell instance as an administrator and open the folder in which you saved the script.
3. Connect to your directory using the `Connect-MgGraph` cmdlet. Here's an example.

```
Connect-MgGraph -TenantId "aaaabbbb-0000-  
cccc-1111-dddd2222eeee" -Scopes  
"Application.Read.All", "User.Read.All"
```

4. Run this PowerShell command.

```
./Get-AzureADPSPPermissions.ps1 | Export-csv -  
Path "Permissions.csv" -NoTypeInfo
```

Disconnect your session with this command.

```
Disconnect-MgGraph
```

Consent is the process of granting authorization to an application to access protected resources on the users' behalf. An administrator or user can be asked for consent to allow access to their organization/individual data.

An application is granted access to data based on a particular user or for the entire organization. Attackers can misuse these consents to gain persistence to the environment and access sensitive data. These types of attacks are called Illicit Consent Grants, which can happen through a phishing email, a user account compromise through password spray, or when an

attacker registers an application as a legitimate user. In scenarios where an administrator account is compromised, then the registration and consent grant are for tenant-wide and not just for one user.

Before an application can access your organization's data, a user must grant the application permissions to do so. Different permissions allow different levels of access. By default, all users are allowed to consent to applications for permissions that don't require administrator consent. For instance, by default, a user can consent to allow an app to access their mailbox but can't consent to allow an app unfettered access to read and write to all files in your organization.

Note

By allowing users to grant apps access to data, users can easily acquire useful applications and be productive. However, in some situations, this configuration can represent a risk if it's not monitored and controlled carefully.

To be able to grant **tenant-wide admin consent**, you must sign in with at least one of the following roles:

- Application Administrator
- Cloud Application Administrator
- **Administrator** - Indicates the consent was provided by the administrator (on behalf of the organization)
- **Individual user** - Indicates the consent was granted by the user and only has access to that user's information
- **Accepted values**
- *AllPrincipals* - Consented by an administrator for the entire tenancy

- *Principal* – Consented by the individual user for data only related to that account

The actual user experience of granting consent differs depending on the policies set on the user's tenant, the user's scope of authority (or role), and the type of permissions requested by the client application. This means that application developers and tenant admins have some control over the consent experience. Admins have the flexibility of setting and deactivating policies on a tenant or app to control the consent experience in their tenant. Application developers can dictate what types of permissions are requested and if they want to guide users through the user consent flow or the admin consent flow.

- **User consent flow** - When an application developer directs users to the authorization endpoint with the intent to record consent for only the current user.
- **Admin consent flow** - When an application developer directs users to the admin consent endpoint with the intent to record consent for the entire tenant. To ensure the admin consent flow works properly, application developers must list all permissions in the **RequiredResourceAccess** property in the application manifest.

Delegated permissions are used by apps that have a signed-in user present and can have consents applied by the administrator or user.

Application permissions are used by apps that run without a signed-in user present. For example, apps that run as background services or daemons. Application permissions can be consented only by an administrator.

For more information, see:

- [Admin consent workflow for admin approval for specific applications](#)
- [Publisher verification program](#)
- [Configure how end users consent to applications](#)

There are thousands (at least) of permissions in the system, and not feasible to list out or parse all of these. The following list addresses commonly misused permissions and others that would create catastrophic impact if misused.

At a high level, Microsoft has observed the following "root" delegated (App+User) permissions being misused in consent phishing attacks. Root equates to the top level. For example, *Contacts.** means to include all delegated permutations of Contacts permissions: *Contacts.Read*, *Contacts.ReadWrite*, *Contacts.Read.Shared*, and *Contacts.ReadWrite.Shared*.

1. *Mail.* (including Mail.Send*, but not Mail.ReadBasic*)*
2. *Contacts.**
3. *MailboxSettings.**
4. *People.**
5. *Files.**
6. *Notes.**
7. *Directory.AccessAsUser.All*
8. *User_Impersonation*

The first seven permissions in the previous list are for Microsoft Graph and the "legacy" API equivalents, such as Outlook REST. The eighth permission is for Azure Resource Manager (ARM) and could also be dangerous on any API that exposes sensitive data with this blanket impersonation scope.

Based on Microsoft Incident Response team's observations, attackers use a combination of the first six permissions in 99% of the consent phishing attacks. Most people don't think of the delegated version of *Mail.Read* or *Files.Read* as a high-risk permission, however, the attacks are generally widespread and target end users, rather than spear phishing against admins who can actually consent to the dangerous permissions. It's recommended to bubble apps with these "critical" level of impact permissions. Even if the applications don't have malicious intent, and if a bad actor were to compromise the app identity, then your entire organization could be at risk.

For the highest risk impact permissions, start here:

- Application permission (AppOnly/AppRole) versions of all the above permissions, where applicable

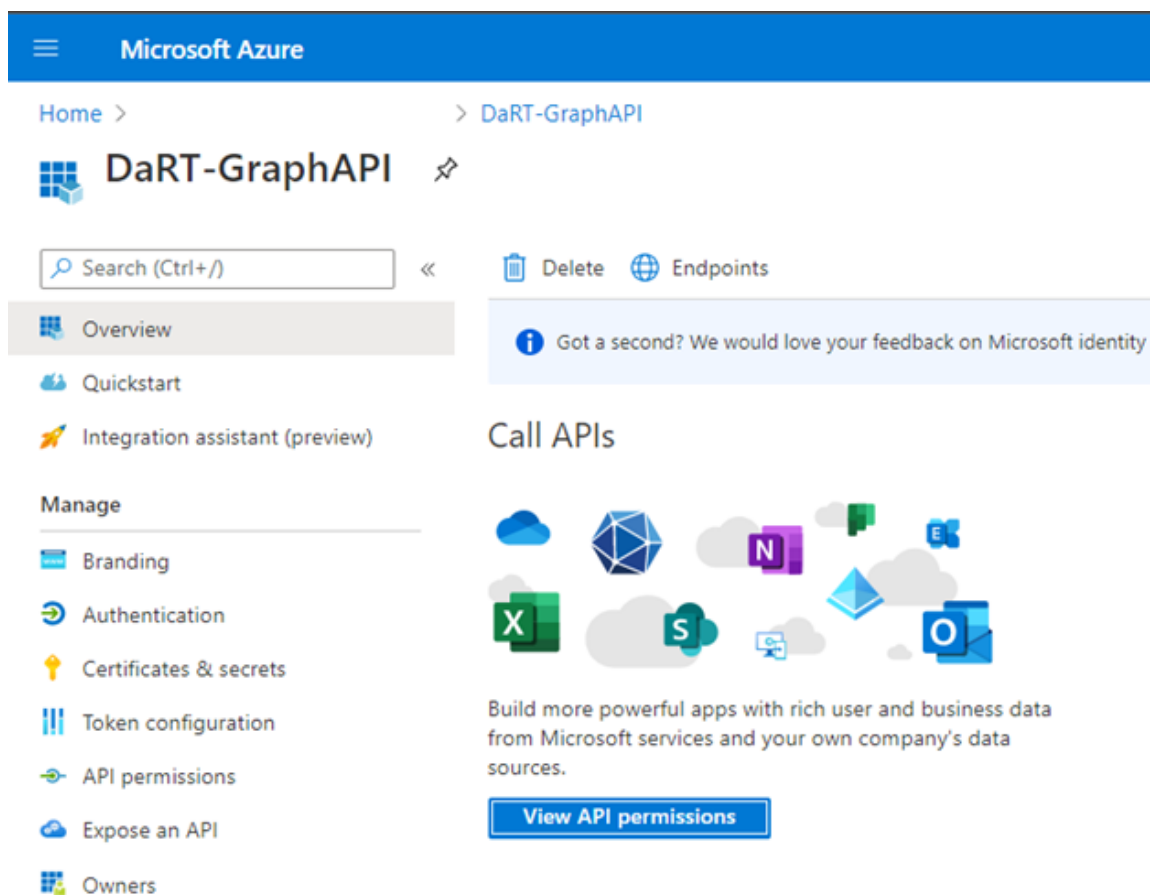
Delegated and AppOnly versions of the following permissions:

- *Application.ReadWrite.All*
- *Directory.ReadWrite.All*
- *Domain.ReadWrite.All**
- *EduRoster.ReadWrite.All**
- *Group.ReadWrite.All*
- *Member.Read.Hidden**
- *RoleManagement.ReadWrite.Directory*
- *User.ReadWrite.All**
- *User.ManageCreds.All*
- All other AppOnly permissions that allow write access

For the lowest risk impact permissions list, start here:

- *User.Read*
- *User.ReadBasic.All*
- *Open_id*
- *Email*
- *Profile*
- *Offline_access* (only if paired with other permissions on this "lowest risk" list)

1. To view the permissions, navigate to the **Registration** screen in the enterprise application.



2. Select **View API permissions**.

Home > Inc. > DaRT-GraphAPI

DaRT-GraphAPI | API permissions

Search (Ctrl+/) « Refresh Got feedback?

- Overview
- Quickstart
- Integration assistant (preview)
- Manage
 - Branding
 - Authentication
 - Certificates & secrets
 - Token configuration
 - API permissions**
 - Expose an API
 - Owners

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

+ Add a permission ✓ Grant admin consent for Inc.

API / Permissions name	Type	Description	Admin consent req...	Status
▼ Microsoft Graph (3)				
AuditLog.Read.All	Application	Read all audit log data	Yes	Granted
Directory.Read.All	Application	Read directory data	Yes	Granted
User.Read	Delegated	Sign in and read user profile	-	Granted

3. Select **Add a permission** and the following screen is displayed.

Request API permissions

Select an API

Microsoft APIs APIs my organization uses My APIs

Commonly used Microsoft APIs



Microsoft Graph

Take advantage of the tremendous amount of data in Office 365, Enterprise Mobility + Security, and Windows 10. Access Azure AD, Excel, Intune, Outlook/Exchange, OneDrive, OneNote, SharePoint, Planner, and more through a single endpoint.



Azure Data Catalog

Programmatic access to Data Catalog resources to register, annotate and search data assets



Azure Data Explorer

Perform ad-hoc queries on terabytes of data to build near real-time and complex analytics solutions



Azure Data Lake

Access to storage and compute for big data analytic scenarios

4. Select **Microsoft Graph** to view the different types of permissions.

Request API permissions

[< All APIs](#)



Microsoft Graph

<https://graph.microsoft.com/> [Docs](#)

What type of permissions does your application require?

Delegated permissions
Your application needs to access the API as the signed-in user.

Application permissions
Your application runs as a background service or daemon without a signed-in user.

Select permissions [expand all](#)

 Start typing a reply url to filter these results

Permission	Admin consent required
> AccessReview	
> AdministrativeUnit	
> Application	
> AppRoleAssignment	
> ApprovalRequest	
> AuditLog (1)	
> BitlockerKey	

5. Select the type of permissions the registered application is using: **Delegated permissions** or **Application permissions**. In the above image, **Application permissions** is selected.
6. You can search for one of the high-risk impact permissions such as **EduRoster**.

Request API permissions



[← All APIs](#)



Microsoft Graph

<https://graph.microsoft.com/> [Docs](#) [↗](#)

What type of permissions does your application require?

Delegated permissions

Your application needs to access the API as the signed-in user.

Application permissions

Your application runs as a background service or daemon without a signed-in user.

Select permissions

[expand all](#)

Permission	Admin consent required
> EduAdministration	
> EduAssignments	
> EduRoster	

7. Select **EduRoster** and expand the permissions.

Request API permissions



[← All APIs](#)



Microsoft Graph

<https://graph.microsoft.com/> [Docs](#) [↗](#)

What type of permissions does your application require?

Delegated permissions

Your application needs to access the API as the signed-in user.

Application permissions

Your application runs as a background service or daemon without a signed-in user.

Select permissions

[expand all](#)

Permission	Admin consent required
> EduAdministration	
> EduAssignments	
▼ EduRoster	
☐ EduRoster.Read.All ⓘ Read the organization's roster	Yes
☐ EduRoster.ReadBasic.All ⓘ Read a limited subset of the	
☐ EduRoster.ReadWrite.All ⓘ Read and write the organization's roster	Yes

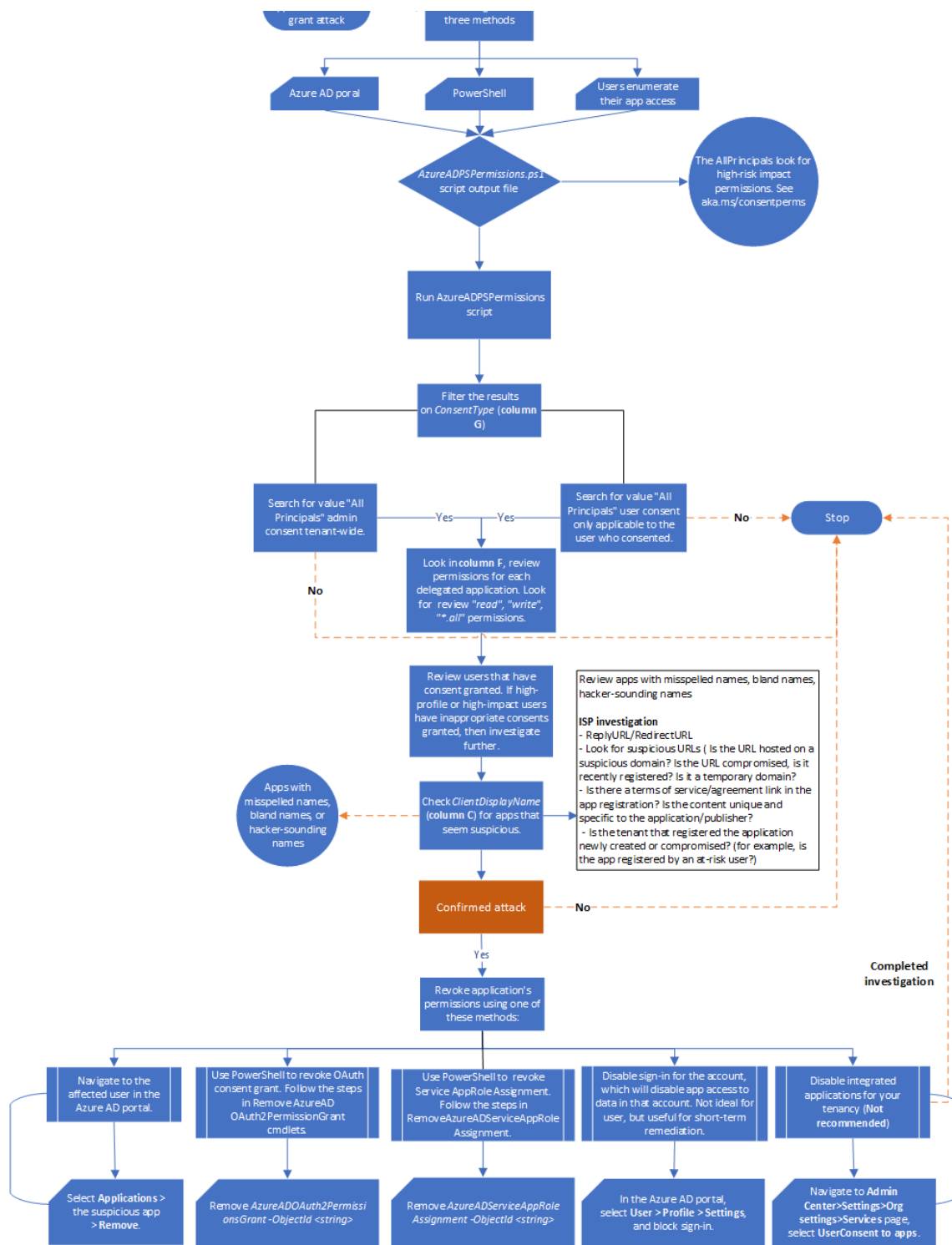
Allows the app to read and write the structure of schools and classes in the organization's roster and education-specific information about all users to be read and written.

8. You can now assign or review these permissions.

For more information, [Graph Permissions](#).

[





]

You can also:

- Download the app consent grant and other incident response playbook workflows as a [PDF](#).
- Download the app consent grant and other incident response playbook workflows as a [Visio file](#).

Use this checklist to perform application consent grant

validation.

- **Indicators of compromise (IoC)**

Check the following indicators of compromise (IoC):

- When did you notice the incident?
- Date range of the incident (how far left is the goal post?)
- Number of compromised accounts
- Names of compromised accounts
- Roles of the compromised accounts
- Are the compromised accounts highly privileged, a standard user, or a combination

- **Roles**

You must be assigned with these roles:

- Local Administrator role on the computer from which you run the script

- **PowerShell configuration**

Configure your PowerShell environment with these steps:

1. Install the Microsoft.Graph PowerShell module.
2. Run the Windows PowerShell app with elevated privileges. (Run as administrator).
3. Configure PowerShell to run signed scripts.
4. Download the [Get-AzureADPSPPermissions.ps1](#) script.

- **Investigation triggers**

- Account compromise
- App Consent settings modified on the tenant
- Alert/audit event status reason "risky application" detected

- Noticed odd looking applications

You can also download the app consent grant and other incident playbook checklists as an [Excel file](#).

You can use the following three methods to investigate application consent grants:

- Azure portal
- PowerShell script
- Microsoft Security Copilot

Note

Using the Azure portal *only allows you to see Admin Consent Grants for the last 90 days and based on this, we recommend using the PowerShell script method only to reduce the attacker registers investigation steps.*

You can use the Microsoft Entra admin center to find applications that individual users have granted permissions.

1. Sign in to the **Azure portal** as an administrator.
2. Select the **Microsoft Entra ID** icon.
3. Select **Users**.
4. Select the user that you want to review.
5. Select **Applications**.
6. You can see the list of apps that are assigned to the user and what permissions these applications have.

There are several PowerShell tools you can use to investigate illicit consent grants, such as:

- HAWK tool
- Microsoft Entra incident response module

- The [Get-AzureADPSPPermissions.ps1](#) script from GitHub

PowerShell is the easiest tool and doesn't require you to modify anything in the tenancy. We're going to base our investigation on the public documentation from the Illicit Consent Grant attack.

Run `Get-AzureADPSPPermissions.ps1`, to export all of the OAuth consent grants and OAuth apps for all users in your tenancy into a `.csv` file. See the [Prerequisites](#) section to download and run the `Get-AzureADPSPPermissions` script.

1. Open a PowerShell instance as an administrator and open the folder where you saved the script.
2. Connect to your directory using the following `Connect-MgGraph` command. Here's an example.

```
Connect-MgGraph -TenantId "aaaabbbb-0000-  
cccc-1111-dddd2222eeee" -Scopes  
"Application.Read.All", "User.Read.All"
```

3. Run this PowerShell command.

```
./Get-AzureADPSPPermissions.ps1 | Export-csv c:  
\temp\consentgrants\Permissions.csv -  
NoTypeInfo
```

4. Once the script completes, it's recommended to disconnect the Microsoft Entra session with this command.

```
Disconnect-MgGraph
```

Note

The script might take hours to complete, depending on the size and permissions configured as well as your connection.

5. The script creates a file named *Permissions.csv*.

- Open the file, then filter or format the data into a table and save as an .xlsx file.

The **column headers** for output are shown in this image.

	A	B	C	D	E	F	G	H	I
1	PermissionType	ClientObjectId	ClientDisplayName	ResourceObjectId	ResourceDisplayName	Permission	ConsentType	PrincipalObjectId	PrincipalDisplayName

- In the **ConsentType** column (G), search for the value **AllPrincipals**. The **AllPrincipals** permission allows the client application to access everyone's content in the tenancy. Native Microsoft 365 applications need this permission to work correctly. ***Every non-Microsoft application with this permission should be reviewed carefully.***
- In the **Permission** column (F), review the permissions that each delegated application has. Look for **Read** and **Write** permission or *. **All** permission, and review these permissions carefully because they might not be appropriate.

Delegated	82fd90a5-24e7-4e26-af38-6286a9c7fa25	UIPath - SPO bot file	841fbc1-cbb6-4ebe-b382-c4c843a1069c	UIPath - SPO bot file	Files.ReadWrite.All	AllPrincipals
-----------	--------------------------------------	-----------------------	-------------------------------------	-----------------------	---------------------	---------------

Note

Review the specific users that have consents granted. If high profile or high impact users have inappropriate consents granted, you should investigate further.

- In the **ClientDisplayName** column (C), look for apps that seem suspicious, such as:

- Apps with misspelled names

Delegated	9139c7ae-44ad-46e9-9f3b-a6a3f2370316	Drivers3dge	841fbc1-cbb6-4ebe-b382-c4c843a1069c	Power BI Service	Dataset.ReadWrite.All	AllPrincipals
-----------	--------------------------------------	-------------	-------------------------------------	------------------	-----------------------	---------------

- Unusual or bland names

Delegated	82fd90a5-24e7-4e26-af38-6286a9c7fa25	Email	841fbc1-cbb6-4ebe-b382-c4c843a1069c	Microsoft Graph	Files.ReadWrite.All	AllPrincipals
-----------	--------------------------------------	-------	-------------------------------------	-----------------	---------------------	---------------

- Hacker-sounding names. You must review these names carefully.

Delegated	9139c7ae-44ad-46e9-9f3b-a6a3f2370316	Intelligence	841fbc1-cbb6-4ebe-b382-c4c843a1069c	Power BI Service	Dataset.ReadWrite.All	AllPrincipals
-----------	--------------------------------------	--------------	-------------------------------------	------------------	-----------------------	---------------

Example Output: AllPrincipals and read write all. Applications might not have anything suspicious like bland names and are using MS graph. However, perform research and determine the

purpose of the applications and the actual permissions the applications have in the tenant, as shown in this example.

PermissionType	ClientObjectId	ClientDisplayName	ResourceObjectId	ResourceDisplayName	Permission	ConsentType	PrincipalObjectId	PrincipalDisplayName
Delegated		Webex Teams Enterprise Content Management		Microsoft Graph	Files.ReadWrite.All	AllPrincipals		
Delegated		Roombelt		Microsoft Graph	Calendars.ReadWrite	AllPrincipals		
Delegated		Lightboard Studio		Microsoft Graph	Files.ReadWrite.AppFolder	AllPrincipals		
Delegated		PRIME Graph API - QA		Microsoft Graph	Calendars.ReadWrite	AllPrincipals		
Delegated		PRIME Graph API - PRD		Microsoft Graph	Calendars.ReadWrite	AllPrincipals		
Delegated		PRIME Graph API - UAT		Microsoft Graph	Calendars.ReadWrite	AllPrincipals		
Delegated		Power BI		Power BI Service	Dataset.ReadWrite.All	AllPrincipals		
Delegated		Power BI		Power BI Service	Dashboard.ReadWrite.All	AllPrincipals		
Delegated		Power BI		Power BI Service	Report.ReadWrite.All	AllPrincipals		
Delegated		Power BI		Power BI Service	Workspace.ReadWrite.All	AllPrincipals		
Delegated		Power BI		Power BI Service	Capacity.ReadWrite.All	AllPrincipals		
Delegated		Power BI		Power BI Service	StorageAccount.ReadWrite.All	AllPrincipals		
Delegated		Power BI		Power BI Service	Dataflow.ReadWrite.All	AllPrincipals		
Delegated		Power BI		Power BI Service	Gateway.ReadWrite.All	AllPrincipals		
Delegated		UIPath - SPO bot file		Microsoft Graph	Files.ReadWrite	AllPrincipals		
Delegated		UIPath - SPO bot file		Microsoft Graph	Files.ReadWrite.All	AllPrincipals		

Here are some useful tips to review information security policy (ISP) investigations:

- ReplyURL/RedirectURL
- Look for suspicious URLs
- Is the URL hosted on a suspicious domain?
- Is it compromised?
- Is the domain recently registered?
- Is it a temporary domain?
- Are there terms of service/service agreement link in the app registration?
- Are the contents unique and specific to the application/publisher?
- Is the tenant that registered the application either newly created or compromised (for example, is the app registered by an at-risk user)?

With [Microsoft Security Copilot](#), you can use natural language prompts to identify and understand risks related to applications or workload identities. This includes the permissions and high privileged permissions granted to them, directly within the Microsoft Entra admin center. Learn more about how to [Assess application risks using Microsoft Security Copilot](#).

While [each attack tends to vary](#), the core attack techniques are:

- An attacker registers an app with an OAuth 2.0 provider, such as Microsoft Entra ID.
- Applications are configured in a way that make them seem legitimate. For example, attackers might use the name of a popular product available in the same ecosystem.
- The attacker gets a link directly from users, which might be done through conventional email-based phishing, by compromising a nonmalicious website, or through other techniques.
- The user selects the link and is shown an authentic consent prompt asking them to grant the malicious app permissions to data.
- If a user selects 'Accept', they grant the app permissions to access sensitive data.
- The app gets an authorization code, which it redeems for an access token, and potentially a refresh token.
- The access token is used to make API calls on behalf of the user.
- If the user accepts, the attacker can gain access to the user's mails, forwarding rules, files, contacts, notes, profile, and other sensitive data and resources.



Permissions requested

Risky App
unverified

This application is not published by Microsoft or your organization.

This app would like to:

- ✓ Maintain access to data you have given it access to
- ✓ Read your contacts
- ✓ Sign you in and read your profile
- ✓ Read your mail
- ✓ Send mail as you
- ✓ Read all OneNote notebooks that you can access
- ✓ Read and write to your mailbox settings
- ✓ Have full access to all files you have access to

Accepting these permissions means that you allow this app to use your data as specified in their terms of service and privacy statement. **The publisher has not provided links to their terms for you to review.** You can change these permissions at <https://myapps.microsoft.com>. [Show details](#)

Cancel

Accept

1. In the Microsoft 365 Defender portal at <https://security.microsoft.com>, go to **Audit**. Or to go directly to the **Audit** page, use <https://security.microsoft.com/auditlogsearch>.
2. On the **Audit** page, search all activities and all users, enter the start date and end date if necessary, and then select **Search**.

Audit log search

Need to find out if a user deleted a document or if an admin reset someone's password? Search the Office 365 audit log to find out what the users and admins in your organization have been doing. You'll be able to find activity related to email, groups, documents, permissions, directory services, and much more. [Learn more about searching the audit log](#)

Search

Clear

Activities

Show results for all activities

Start date

2020-04-16 00:00

End date

2020-07-16 00:00

Users

Show results for all users

File, folder, or site

Add all or part of a file name, folder name, or URL

Results 150 results found (More items available, scroll down to see more.)

Filter results

Export results

Date	IP address	User	Activity	Item	Detail
2020-07-15 16:01:34		Service Account	SearchHttpStatus		
2020-07-15 14:40:37		00aa00aa-bb11-cc22-dd33-44ee44ee44ee	SearchCustomerInsight		
2020-07-15 14:40:24		00aa00aa-bb11-cc22-dd33-44ee44ee44ee	SearchCustomerInsight		
2020-07-15 14:40:23		Service Account	SearchMailFlowStatusSummary		
2020-07-15 14:40:20		00aa00aa-bb11-cc22-dd33-44ee44ee44ee	SearchCustomerInsight		
2020-07-15 14:40:15		Service Account	SearchAggCompromiseReport		
2020-07-15 14:40:09		Service Account	SearchAggTPSReportData		
2020-07-15 14:40:06		Service Account	AggregateMailMetadata		

3. Select **Filter** results and in the **Activity** field, enter **Consent** to application.

Audit log search

Need to find out if a user deleted a document or if an admin reset someone's password? Search the Office 365 audit log to find out what the users and admins in your organization have been doing. You'll be able to find activity related to email, groups, documents, permissions, direct

[Learn more about searching the audit log](#)

Search Clear Results 150 results found (More items available, scroll down to see more.) Hide filtering

Activities Show results for all activities

Start date 2020-04-16 00:00

Date	IP address	User	Activity	Item	Detail
			Consent to applications		

4. If you have activity under consent to grant, continue to the next step.
5. Select the result to see the details of the activity. Select **More Information** to get details of the activity.
6. Check whether IsAdminContent is set to 'True'.

Note

This process can take from 30 minutes up to 24 hours for the corresponding audit log entry to be displayed in the search results after an event occurs.

The extent of time that an audit record is retained and is searchable in the audit log depends on your Microsoft 365 subscription, and specifically the type of the license that is assigned to a specific user. **If this value is true, it indicates that someone might have granted broad access to data. If this is unexpected, take immediate steps to confirm an attack.**

If you have one or more instances of the IoCs previously listed, you need to do further investigation to positively confirm that the attack occurred.

You can inventory apps for your users using the Microsoft Entra admin center, PowerShell, or have your users individually enumerate their application access.

- Use the Microsoft Entra admin center to inventory applications and their permissions. This method is thorough, but you can only check one user at a time, which can be time-consuming if

you have to check the permissions of several users.

- Use PowerShell to inventory applications and their permissions. This method is the fastest and most thorough, with the least amount of overhead.
- Encourage your users to individually check their apps and permissions and report the results back to the administrators for remediation.

You can use the Microsoft Entra admin center to see the list of apps to which individual users granted permissions.

1. Sign in to the **Azure Portal** with administrative rights.
2. Select the **Microsoft Entra ID** icon.
3. Select **Users**.
4. Select the user that you want to review.
5. Select **Applications**.

You can see the list of apps that are assigned to the user and the permissions granted to these apps.

After finishing inventorying application access, review the audit log to determine the full scope of the breach. Search on the affected users, the time frames that the illicit application had access to your organization, and the permissions the app had. You can search the audit log in the Microsoft 365 Security & Microsoft Purview compliance portals.

Important

If auditing wasn't enabled before the possible attack, you **won't** be able to investigate because auditing data isn't available.

- Regularly [audit applications](#) and granted permissions in your organization to ensure no unwarranted or suspicious

applications are granted access to data.

- [Review, detect, and remediate illicit consent grants in Office 365](#). for more best practices and safeguards against suspicious applications requesting OAuth consent.

If your organization has the appropriate license:

- Use more [OAuth application](#) auditing features in Microsoft Defender for Cloud Apps.
- Use [Azure Monitor Workbooks](#) to monitor permissions and consent related activity. The Consent Insights workbook provides a view of apps by number of failed consent requests. This can be helpful to prioritize applications for administrators to review and decide whether to grant them admin consent.

After identifying an application with illicit permissions, **immediately disable the application** following the instructions in [Disable an application](#). Then, contact Microsoft Support to report the malicious application.

Once an application is disabled in Microsoft Entra, it can't obtain new tokens to access data, and other users can't sign in or grant consent to the app.

Note

If you suspect you have encountered a malicious application in your organization, it's better to disable it than to delete it. If you only delete the application, it might return later if another user grants consent. Instead, disable the application to ensure it can't come back later.

There are various consent attack types, these recommended defenses mitigate all types of attacks, especially consent phishing, where attackers trick users into granting a malicious app access to sensitive data or other resources. Instead of

trying to steal the user's password, an attacker is seeking permission for an attacker-controlled app to access valuable data.

To help prevent consent attacks from affecting Microsoft Entra ID and Office 365, see the following recommendations:

- This setting has user implications and might not be applicable for an environment. If you're going to allow any consents, ensure the administrators approve the requests.
- Allow consents for applications from verified publishers only and specific types of permissions classified as low impact.

Note

The above recommendations are suggested based on the most ideal, secure configurations. However, as security is a fine balance between functionalities and operations, the most secure configurations might cause more overhead to administrators. It's a decision best made after consulting with your administrators.

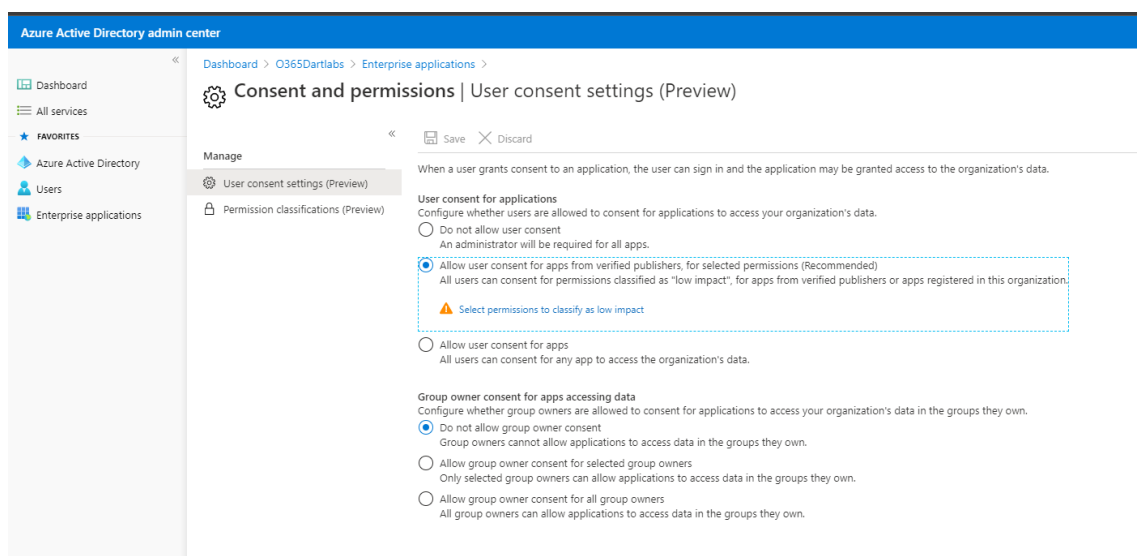
Configure risk-based step-up consent - Enabled by default if user consent to grants is enabled

- Risk-based step-up consent helps reduce user exposure to malicious apps that make illicit consent requests. If Microsoft detects a risky end-user consent request, the request requires a "step-up" to admin consent instead. This capability is enabled by **default**, but it only results in a behavior change when **end-user consent is enabled**.
- When a risky consent request is detected, the consent prompt displays a message indicating that admin approval is needed. If the admin consent request workflow is enabled, the user can send the request to the admin for further review directly from the consent prompt. If enabled, the following message is displayed:

*AADSTS90094: <clientAppDisplayName> needs permission to access resources in your organization that only an admin can grant. Please ask an admin to grant permission to this app before you can use it. In this case, an audit event will also be logged with a Category of "**ApplicationManagement**" Activity Type of "**Consent to application**", and Status Reason of "**Risky application detected**".*

Note

Any tasks that require administrator's approval has operational overhead. The "**Consent and permissions, User consent settings**" is in **Preview** currently. Once it's ready for general availability (GA), the "**Allow user consent from verified publishers, for selected permissions**" feature should reduce administrators' overhead and it's recommended for most organizations.



Educate your application developers to follow the trustworthy app ecosystem. To help developers build high-quality and secure integrations, we're also announcing [public preview of the Integration Assistant in Microsoft Entra app registrations](#).

- The Integration Assistant analyzes your app registration and

benchmarks it against a set of recommended security best practices.

- The Integration Assistant highlights best practices that are relevant during each phase of your integration's lifecycle—from development all the way to monitoring—and ensures every stage is properly configured.
- It makes your job easier, whether you're integrating your first app or you're an expert looking to improve your skills.

Educate your organization on consent tactics ([phishing tactics](#), [admin and user consents](#)):

- Check for poor spelling and grammar. If an email message or the application's consent screen has spelling and grammatical errors, it's likely to be a suspicious application.
- Keep a watchful eye on app names and domain URLs. Attackers like to spoof app names that make it appear to come from legitimate applications or companies but drive you to consent to a malicious app.
- Make sure you recognize the app name and domain URL before consenting to an application.
- Promote the use of applications that are publisher verified. Publisher verification helps admins and end users understand the authenticity of application developers. Over 660 applications by 390 publishers are verified thus far.
- Configure application consent policies by allowing users to only consent to specific applications you trust, such as applications developed by your organization or from verified publishers.
- Educate your organization on how our permissions and consent [framework works](#).

- Understand the data and permissions an application is asking for and understand how permissions and consent work within our platform.
- Ensure administrators know how to manage and evaluate consent requests.

Audit apps and consented permissions in your organization to ensure applications being used are accessing only the data they need and adhering to the principles of least privilege.

- Educate the customer and provide awareness and training on securing application consent grants
- Tighten the application consent grants process with organizational policy and technical controls
- Set up **Create schedule** to review **Consented** applications
- You can use PowerShell to disable suspected or malicious apps by [disabling the app](#)
- [Protecting remote workforce application attacks](#)
- [Fostering a secure and trustworthy app ecosystem](#)
- [Investigate risky OAuth apps](#)
- [Managing consent to applications and evaluating consent requests](#)
- [Disable user sign-ins for an enterprise app in Microsoft Entra ID](#)
- [Understand the permissions and consent framework in the Microsoft identity platform.](#)
- [Understand the difference between delegated permissions and application permissions.](#)
- [Configure how end-users consent to applications](#)

- [Unexpected application in my applications list](#)
- [Detect and Remediate Illicit Consent Grants](#)
- [How and Why Microsoft Entra Applications are Added](#)
- [Application and service principal objects in Microsoft Entra ID](#)
- [Microsoft Entra Config Documentor](#)
- [Managing consent to applications and evaluating consent requests](#)
- [Get-MgServicePrincipal](#)
- [Build 2020: Fostering a secure and trustworthy app ecosystem for all users](#)
- [Configure the admin consent workflow](#)
- [Admins should evaluate all consent requests carefully before approving a request.](#)
- [Application Registration vs. Enterprise Applications](#)
- [Permissions](#)
- [KrebsOnSecurity on AppConsent Phishing](#)

Examine guidance for identifying and investigating these additional types of attacks:

- [Phishing](#)
- [Password spray](#)
- [Overview](#) for Microsoft security products and resources for new-to-role and experienced analysts
- [Planning](#) for your Security Operations Center (SOC)
- [Microsoft Defender XDR](#) incident response
- [Microsoft Defender for Cloud \(Azure\)](#)

- [Microsoft Sentinel](#) incident response
- [Microsoft Incident Response team guide shares best practices for security teams and leaders](#)
- [Microsoft Incident Response guides help security teams analyze suspicious activity](#)